Reductions and NP-completeness

Exercise sheet 9 & 10 (solutions)

Throughout, graphs are finite, simple and undirected. A verifier first checks that its input and certificate have the required form; malformed inputs are rejected. In reductions, a malformed source instance is sent to a fixed no-instance of the target problem.

Exercise 9.1. Recall that $L^* = \{w_1 \cdots w_r \mid r \geq 0,\ w_i \in L\}$. Suppose $L \in \mathrm{NP}$. Is it true that $L^* \in \mathrm{NP}$? Prove your answer.

Solution. Yes. Let V verify membership in L, with certificate length at most $p(n)$ and running time at most $q(n)$ on an input pair of length n. We may take p and q to be nondecreasing polynomials.

The empty word belongs to L^*, even when L is empty, so accept it immediately. For a nonempty word w of length n, a certificate gives a division

$$w = w_1 \cdots w_r$$

into nonempty factors, together with a certificate s_i for each w_i. The verifier checks the division, checks $|s_i| \leq p(|w_i|)$, and runs $V(w_i, s_i)$ for every i. It accepts exactly when all these checks succeed.

There are at most n factors: each consumes at least one letter of w. Thus the cuts and certificates use at most $O(n \log(n+1) + np(n))$ symbols. The verifier makes at most n calls on pairs of length at most $O(n + p(n))$, taking at most $nq(O(n + p(n)))$ steps, apart from polynomial time for parsing and checking the cuts.

If $w \in L^*$, discard any empty factors from a factorisation of w and supply certificates for the remaining factors. The verifier accepts. Conversely, an accepted certificate expresses w as a concatenation of words in L. Therefore $L^* \in \mathrm{NP}$. □

Exercise 9.2. Prove that INDEPENDENT-SET is NP-complete, where

Input: A finite undirected graph G and a non-negative integer k encoded in binary.

Question: Does G contain a set of at least k vertices with no edge between any two of them?

Solution. A certificate lists k distinct vertices. The verifier first rejects if $k > |V(G)|$, and otherwise checks that the listed vertices belong to G and that no pair is adjacent. The list has polynomial length, and checking all pairs takes polynomial time. Thus INDEPENDENT-SET $\in$ NP.

Reduce CLIQUE, whose NP-completeness was proved in Lecture 9, to INDEPENDENT-SET. Given (G, k), construct the complement graph $\overline{G}$: keep the vertices, and put an edge between two distinct vertices exactly when there is no edge between them in G. Output $(\overline{G}, k)$.

$$S \text{ is a clique in } G \iff S \text{ is an independent set in } \overline{G}.$$

Thus, the answer is preserved, including when k exceeds the number of vertices. Constructing the complement requires checking at most $|V(G)|^2$ pairs; even with repeated scans of the input, this takes polynomial time in its length. Therefore INDEPENDENT-SET is NP-hard and hence NP-complete. □

Exercise 9.3. Prove that VERTEX-COVER is NP-complete, where

Input: A finite undirected graph G and a non-negative integer k encoded in binary.

Question: Does G have a set of at most k vertices that contains at least one endpoint of every edge?

Solution. A certificate specifies a subset C of the vertices, using one bit per vertex. The verifier checks $|C| \leq k$ and checks that every edge has an endpoint in C. The certificate has polynomial length, and the checks take polynomial time, so VERTEX-COVER $\in$ NP.

Reduce INDEPENDENT-SET to VERTEX-COVER. Given (G, k), let $n = |V(G)|$. If $k > n$, output a single edge with bound 0, a fixed no-instance. Otherwise, output $(G, n - k)$.

For every $S \subseteq V(G)$,

$$S \text{ is independent} \iff V(G) \setminus S \text{ is a vertex cover.}$$

Indeed, an edge not covered by $V(G) \setminus S$ would have both endpoints in S, and conversely. Moreover, $|S| \geq k$ exactly when $|V(G) \setminus S| \leq n - k$. This proves that the reduction preserves the answer. Copying G and performing the binary subtraction take polynomial time. Hence VERTEX-COVER is NP-complete. □

Exercise 9.4. Prove that HITTING-SET is NP-complete, where

Input: A finite set U and a family $S_1, \ldots, S_m \subseteq U$, with the elements and sets explicitly listed, and a non-negative integer k encoded in binary.

Question: Is there a set $H \subseteq U$ with $|H| \leq k$ such that $H \cap S_i \neq \emptyset$ for every $i \in \{1, \ldots, m\}$?

Solution. A certificate selects elements of U, using one bit per listed element. The verifier checks that at most k are selected and that each S_i contains a selected element. The lists are explicit, so these checks take polynomial time. Thus HITTING-SET $\in$ NP.

Reduce VERTEX-COVER to HITTING-SET. Given (G, k), take

$$U = V(G), \qquad \mathcal{S} = \{\{u, v\} \mid \{u, v\} \in E(G)\},$$

and retain k. A set $H \subseteq U$ intersects every member of $\mathcal{S}$ exactly when it contains an endpoint of every edge, that is, when it is a vertex cover. The size bound is the same. Listing one two-element set per edge takes polynomial time. Therefore HITTING-SET is NP-complete. □

Exercise 9.5. Prove that DOUBLE-SAT is NP-complete, where

Input: A Boolean formula φ, encoded as a word.

Question: Does φ have at least two distinct satisfying assignments?

Solution. A certificate consists of two assignments to the variables occurring in φ. The verifier checks that they differ on at least one variable and evaluates φ on both. Each assignment has at most $|\varphi|$ bits, and both evaluations take polynomial time. Hence DOUBLE-SAT $\in$ NP.

Reduce SAT to DOUBLE-SAT. Choose a fresh variable z and output

$$\psi = \varphi \wedge (z \vee \neg z).$$

Every satisfying assignment of φ has two distinct extensions satisfying ψ, one with $z = T$ and one with $z = F$. If ψ has a satisfying assignment, its restriction to the original variables satisfies φ. Consequently,

$$\varphi \text{ is satisfiable} \iff \psi \text{ has at least two satisfying assignments.}$$

The fresh variable really occurs in ψ, so these extensions are distinct assignments to its variables. Finding a fresh name and adding the tautological clause take polynomial time. This proves NP-hardness and hence NP-completeness. □

Exercise 9.6. Prove that 3SAT is NP-complete, where

Input: A CNF formula with exactly three literal occurrences in every clause. A literal may occur more than once in a clause.

Question: Is the formula satisfiable?

Solution. A certificate gives a truth value for every variable occurring in the formula. The verifier checks the three-literal format and evaluates the clauses. There are at most as many variables and literal occurrences as symbols in the input, so the certificate and verification time are polynomially bounded.

For hardness, reduce CNF-SAT to 3SAT. We replace each clause separately, using fresh variables for different clauses. Repeat a literal to extend a clause of length one or two to length three. Leave clauses of length three unchanged. For $r \geq 4$, replace $\ell_1 \vee \cdots \vee \ell_r$ by

$$\begin{aligned}&(\ell_1 \vee \ell_2 \vee y_1)\\ &\quad \wedge \bigwedge_{j=1}^{r-4} (\neg y_j \vee \ell_{j+2} \vee y_{j+1})\\ &\quad \wedge (\neg y_{r-3} \vee \ell_{r-1} \vee \ell_r).\end{aligned}$$

There are $r-3$ new variables and $r-2$ clauses. For $r = 4$, the middle conjunction is empty.

If all the original literals are false, the first clause forces $y_1 = T$, the next forces $y_2 = T$, and so on; the last clause then fails. If at least one original literal is true, set

$$y_j = T \iff \ell_1, \ldots, \ell_{j+1} \text{ are all false.}$$

Every new clause is then true. Thus, an assignment satisfies the original clause exactly when it can be extended to satisfy its replacement. Since different clauses use different new variables, this equivalence holds for the entire formula.

An empty clause makes the input unsatisfiable; send it to $(z \vee z \vee z) \wedge (\neg z \vee \neg z \vee \neg z)$. An empty conjunction is sent to the satisfiable formula $(z \vee \neg z \vee z)$.

If the input length is N, the construction adds $O(N)$ clauses and variables. Giving the new variables binary indices produces an output of length $O(N \log(N+1))$; sequential scans suffice to construct it in polynomial time. Thus CNF-SAT $\leq_p$ 3SAT, proving NP-completeness. □

Exercise 9.7. * Prove that SUBSET-SUM is NP-complete, where

Input: A list of non-negative integers $a_1, \ldots, a_m$ and a non-negative integer t. All integers are encoded in binary.

Question: Is there an index set $I \subseteq \{1, \ldots, m\}$ such that

$$\sum_{i \in I} a_i = t?$$

Solution. A certificate specifies the index set I. The verifier adds the corresponding integers and checks whether their sum is t. If there are m integers and each has at most b bits, their sum has at most $b + \lceil \log_2(m+1) \rceil$ bits. Binary addition and comparison take polynomial time, so SUBSET-SUM belongs to NP.

For hardness, reduce 3SAT to SUBSET-SUM using decimal columns. If the formula has no clauses, output the fixed yes-instance with list (1) and target 1. Otherwise, let its variables be $x_1, \ldots, x_n$ and its clauses be $C_1, \ldots, C_m$. Use $n + m$ decimal columns: one for each variable and one for each clause.

For each variable x_i, create two numbers A_i^T and A_i^F. Both have digit 1 in variable column i and 0 in the other variable columns. In clause column j, put

number	digit in column C_j
A_i^T	number of occurrences of x_i in C_j
A_i^F	number of occurrences of $\neg x_i$ in C_j

Counting occurrences handles repeated literals. For each clause C_j, add two numbers that have digit 1 in its column and 0 in every other column. Set the target to

$$\underbrace{11\cdots1}_{n \text{ variable columns}} \quad \underbrace{33\cdots3}_{m \text{ clause columns}} .$$

No addition produces a carry: a variable column totals at most 2, and a clause column at most $3 + 2 = 5$, even if every number is selected.

To reach the target, exactly one of A_i^T and A_i^F must be selected for each i, so these choices specify a truth assignment. In clause column j, the selected variable numbers contribute the number of true literal

occurrences in C_j. The two extra numbers can raise this sum to 3 exactly when it is 1, 2, or 3; they cannot raise 0 to 3. Thus, the target can be reached exactly when every clause is satisfied.

There are $2n+2m$ numbers with at most $n+m$ decimal digits each, so each has $O(n+m)$ binary bits and the whole output has polynomial length. Constructing the numbers in binary also takes polynomial time. This is a polynomial-time reduction, proving NP-hardness. Therefore SUBSET-SUM is NP-complete. □

Exercise 9.8. * Prove that KNAPSACK is NP-complete, where

Input: A list of pairs $(w_1, v_1), \ldots, (w_n, v_n)$ of non-negative integers, and non-negative integers W and V. All integers are encoded in binary.

Question: Is there an index set $I \subseteq \{1, \ldots, n\}$ such that

$$\sum_{i \in I} w_i \leq W \qquad \text{and} \qquad \sum_{i \in I} v_i \geq V ?$$

Solution. A certificate specifies the selected index set I, using one bit for each item. The verifier computes the total weight and value and checks the two inequalities. If there are n items and each input integer has at most b bits, either sum has at most $b + \lceil \log_2(n+1) \rceil$ bits. The sums and comparisons can therefore be computed in polynomial time, so KNAPSACK belongs to NP.

For hardness, reduce SUBSET-SUM to KNAPSACK. Given non-negative integers $a_1, \ldots, a_m$ and target t, make item i have weight $w_i = a_i$ and value $v_i = a_i$. Set the weight capacity and target value to $W = t$ and $V = t$.

For any index set $I \subseteq \{1, \ldots, m\}$, the selected total weight and value are both $\sum_{i \in I} a_i$. Hence

$$\sum_{i \in I} a_i = t \iff \left(\sum_{i \in I} w_i \leq W \quad \text{and} \quad \sum_{i \in I} v_i \geq V \right).$$

The constructed KNAPSACK instance is therefore a yes-instance exactly when the SUBSET-SUM instance is. The construction copies the input numbers and writes t twice, so it takes polynomial time and produces an output of polynomial length. Since SUBSET-SUM is NP-complete by Exercise 9.7, KNAPSACK is NP-hard. Together with membership in NP, this proves that it is NP-complete. □

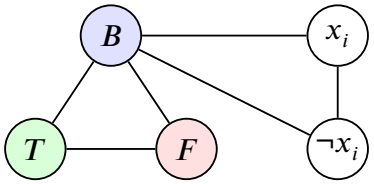
Exercise 9.9. * Prove that 3COLOUR is NP-complete, where

Input: A finite undirected graph G.

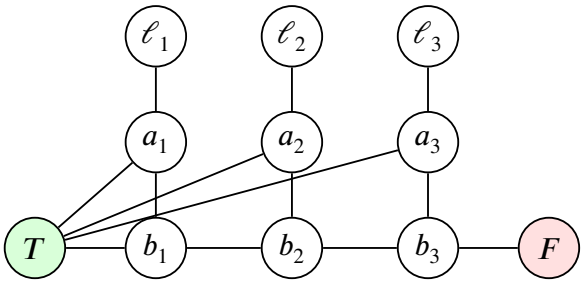
Question: Can the vertices of G be coloured with at most three colours so that the endpoints of every edge have different colours?

Solution. A certificate assigns one of three colours to each vertex. Checking every edge takes polynomial time, so 3COLOUR $\in$ NP. We reduce 3SAT to 3COLOUR.

The two gadgets are shown below.



The gadget on the left: palette and variable.



The gadget on the right: one clause.

The gadget on the left: truth values. Its triangle has vertices T, F, B. Their three colours must differ; name the colours after these vertices. For each variable x_i, the two adjacent vertices x_i and $\neg x_i$ are both joined to B. They must receive opposite colours T and F, so their colours specify a truth value for x_i and its negation.

The gadget on the right: clauses. For a clause $\ell_1 \vee \ell_2 \vee \ell_3$, use the corresponding literal vertices and six fresh vertices $a_1, a_2, a_3, b_1, b_2, b_3$. Join ℓ_i to a_i, a_i to b_i, and a_i to the palette vertex T for each i; also add the path from T through b_1, b_2, b_3 to F. The vertices T and F are the same palette vertices from the gadget on the left in every copy. Repeated literals use the same literal vertex but different auxiliary vertices.

If all three literals have colour F, each a_i is adjacent to both colours T and F, so all a_i have colour B. The b_i can then use only T and F. Along the path in the gadget on the right their colours must alternate: starting at T, four edges bring us back to colour T, contradicting the endpoint F.

If ℓ_r has colour T, give a_r colour F and the other a_i colour B. The following choices complete the colouring:

r	b_1	b_2	b_3
1	B	F	T
2	F	B	T
3	F	T	B

Check the path edges and the three edges $a_i b_i$: their endpoints have different colours. Consequently, the clause graph can be coloured exactly when at least one literal is true.

The whole graph. Use one palette, one pair of vertices per variable, and a fresh clause graph per clause. A satisfying assignment colours the literal vertices, and the table above extends it independently through every clause graph. Conversely, any proper colouring gives a truth assignment, and the clause argument shows that it satisfies every clause.

For n variables and m clauses there are $3 + 2n + 6m$ vertices and $3 + 3n + 13m$ edges. The graph is constructed by listing these vertices and edges, so the construction takes polynomial time in the formula length. Therefore 3SAT $\leq_p$ 3COLOUR, proving NP-completeness. □

Exercise 9.10. * Prove that HAMILTONIAN-CYCLE is NP-complete, where

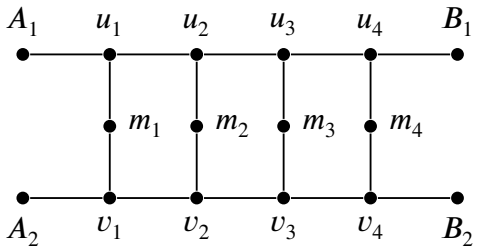
Input: A finite undirected graph G.

Question: Does G have a cycle that visits every vertex exactly once before returning to its starting vertex?

Solution. A certificate lists the vertices in cyclic order. The verifier checks that the graph has at least three vertices, that each vertex occurs exactly once in the list, and that consecutive vertices, including the last and first, are adjacent. The certificate has polynomial length and these checks take polynomial time. Thus HAMILTONIAN-CYCLE $\in$ NP.

We give an English adaptation of Alexeev's reduction from 3SAT, with the small graphs and their behaviour explained explicitly. The construction uses two graphs: one couples two choices, and the other expresses the condition imposed by a clause.

The α-graph: exactly one of two connections. The graph below has twelve internal vertices: u_i, m_i, v_i for $1 \le i \le 4$. Only the four displayed connections to the outside vertices A_1, B_1, A_2, B_2 are allowed.



Every m_i has degree two, so a Hamiltonian cycle must use both edges $u_i m_i$ and $m_i v_i$. Each u_i and v_i must therefore use exactly one horizontal edge as well.

On the upper row there are only two possibilities: use u_1u_2 and u_3u_4, with neither outside connection, or use A_1u_1, u_2u_3, and u_4B_1. The same applies to the lower row. Choosing the first possibility on both rows creates separate internal cycles; choosing the second on both rows creates an internal cycle through rungs 2 and 3. Neither can be part of a single Hamiltonian cycle through the whole graph. Thus one row uses its outside connections and the other does not.

In the upper-connection case, the traversal is

$$A_1, u_1, m_1, v_1, v_2, m_2, u_2, u_3, m_3, v_3, v_4, m_4, u_4, B_1.$$

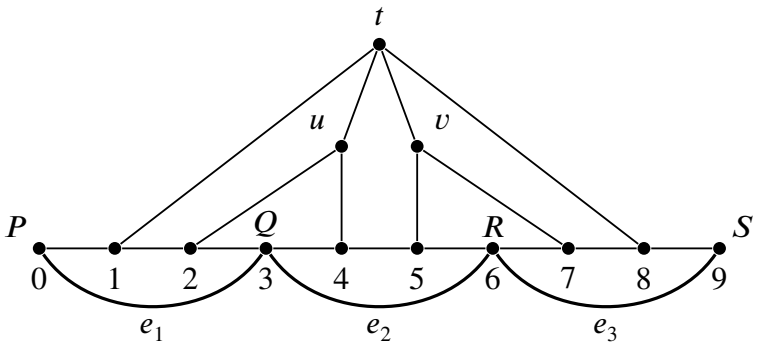
The other traversal is obtained by interchanging u and v, and the subscripts 1 and 2 on the outside vertices. Both visit all twelve internal vertices. We can therefore regard the graph as two possible connections A_1B_1 and A_2B_2, of which a Hamiltonian cycle must use *exactly one*. It cannot enter on one row and leave on the other.

The β-graph: at least one of three edges is omitted. In the next graph, let $P = 0$, $Q = 3$, $R = 6$, and $S = 9$. The three thick edges are

$$e_1 = PQ, \qquad e_2 = QR, \qquad e_3 = RS.$$

Besides these edges, the graph contains the path $0, 1, \ldots, 9$ and edges

$$t1,\ t8,\ tu,\ tv,\ u2,\ u4,\ v5,\ v7.$$



Only P and S will have connections to the rest of the construction. A Hamiltonian cycle must therefore visit this graph along a path from P to S that includes every one of its vertices.

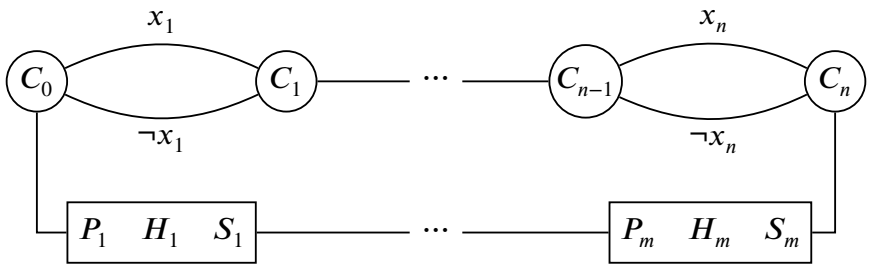
Such a path cannot use all three thick edges: they would already form the whole path P, Q, R, S, leaving the other vertices unvisited. Every other subset of thick edges is possible. Here are all seven cases; each row lists a path visiting every vertex once:

thick edges used	path from 0 to 9
$\emptyset$	$0, 1, 2, 3, 4, u, t, v, 5, 6, 7, 8, 9$
$\{e_1\}$	$0, 3, 4, u, 2, 1, t, v, 5, 6, 7, 8, 9$
$\{e_2\}$	$0, 1, 2, 3, 6, 5, 4, u, t, v, 7, 8, 9$
$\{e_3\}$	$0, 1, 2, 3, 4, u, t, 8, 7, v, 5, 6, 9$
$\{e_1, e_2\}$	$0, 3, 6, 5, 4, u, 2, 1, t, v, 7, 8, 9$
$\{e_1, e_3\}$	$0, 3, 4, u, 2, 1, t, 8, 7, v, 5, 6, 9$
$\{e_2, e_3\}$	$0, 1, 2, u, t, 8, 7, v, 5, 4, 3, 6, 9$

Arranging the variable choices and clauses. Let $\varphi = C_1 \wedge \cdots \wedge C_m$ be a 3CNF over variables $x_1, \ldots, x_n$. Each literal occurrence will be treated separately, even when literals repeat. If there are no clauses, output a triangle. Otherwise retain only variables that occur in the formula.

First describe a graph before inserting the α-graphs. Take vertices $C'_0, C'_1, \ldots, C'_n$ and put two parallel edges between C'_{i-1} and C'_i, labelled x_i and $\neg x_i$. They represent the two possible truth values. Also take a copy H_j of the β-graph for each clause C_j, labelling its three thick edges by the three literal occurrences of that clause. Join S_j to P_{j+1}, join C'_0 to P_1, and join S_m to C'_n.

The diagram shows this arrangement, with the primes on the C'_i omitted to keep the labels short. Each box is a complete copy of the β-graph:



A cycle through this arrangement chooses one edge for each variable and passes through every clause graph. We now couple each literal edge on the variable side to every occurrence of that literal on the clause side, using α-graphs. Their exactly-one property will make a clause edge omitted precisely when the corresponding literal is true.

Inserting the α-graphs. Here are the precise steps, including when a literal occurs several times. Suppose a literal ℓ has $k > 0$ occurrences. Replace its variable edge by the upper row of k successive α-graphs: each row has four fresh vertices, and the last upper vertex of one copy is joined directly to the first upper vertex of the next. Attach the two ends to C'_{i-1} and C'_i. Do not insert extra vertices between copies.

For each occurrence of ℓ, replace its thick clause edge by the lower row of the corresponding copy, again with four fresh vertices between the original endpoints. Add the four middle vertices and their vertical edges. This completes that α-graph. If a literal has no occurrences, leave its variable edge unchanged.

The exactly-one property makes all upper connections along a given variable route either used or unused: two consecutive copies share the edge joining their upper rows. If this route is used, all its occurrence edges are omitted on the clause side; if it is unused, they are all used. The internal vertices are visited in either case, by the appropriate zigzag traversal.

The final graph is simple. Every retained variable occurs somewhere, so at least one of its two parallel edges has been replaced by a route through new vertices. All other new vertices are private to their copies.

Why the answer is preserved. Suppose φ has a satisfying assignment. For each variable, choose the route labelled by its true literal. In each clause graph use exactly the thick edges whose literals are false. At least one literal is true, so at least one thick edge is omitted, and the table of seven paths provides the required traversal. Expand every coupled pair into its corresponding α traversal. These pieces join into one cycle and visit every vertex exactly once.

Conversely, suppose the constructed graph has a Hamiltonian cycle. Replace every α traversal by the connection it uses. The resulting cycle passes along the variable chain, choosing one route per variable, and through each β-graph from P_j to S_j. Set x_i true exactly when the route labelled x_i is chosen. In each clause graph, at least one thick edge is omitted. Its α-graph therefore uses the corresponding variable route, so that literal is true. Every clause is satisfied.

There are m clause graphs and one α-graph for each of the $3m$ literal occurrences. The final graph has $n + 1 + 13m + 36m$ vertices and $O(n + m)$ edges. Listing them and their incidences takes polynomial time in the formula length. This gives 3SAT $\leq_p$ HAMILTONIAN-CYCLE and proves NP-completeness. □

Sources. The Hamiltonian-cycle construction is adapted from V. B. Alexeev, *Introduction to the Theory of Algorithm Complexity* (2002), §4.8, pp. 60–63; the two gadgets are redrawn from his Figures 1–2. The supplied edition does not contain the 3-colouring construction. For that construction, see Anup Rao, Lecture 8, Figure 2; see also UW CSE421, Lecture 25 for the reduction strategy. The HITTING-SET reduction is also given in Toronto CSC373, Assignment 3 solutions, Question 6(a). For clause splitting and the numerical reductions, see Paul W. Goldberg, Computational Complexity, slides 5.